

PROBING TRANSFORMER REASONING TRANSFER THROUGH ALPHABET EQUIVALENCE

Anonymous authors

Paper under double-blind review

ABSTRACT

A reasoner is *abstract* if it applies a general procedure to a problem’s structure, indifferent to how that structure is concretely instantiated; it is *concrete* if what it learned is tied to the particular instantiation trained on. This gives a testable marker: an abstract reasoner should transfer mastery of one problem instance to another. We study equivalence via alphabet renaming, and test transfer using a weight-transplant control (copying trained per-symbol weights onto the new instance without retraining) that distinguishes true reasoning failure from symbols never trained on. We instantiate this with Sudoku, where an equivalent puzzle is any relabeling of its digits. A transformer trained on one such puzzle transfers to an equivalent one no better than an untrained network, yet the transplant control recovers its original performance exactly. Reasoning, in that narrow sense, is abstract; recognizing that a new instance calls for it is not. We probe this recognition gap three ways. Freezing the shared network and adapting only a new alphabet’s parameters fares worse than training an unfrozen model from scratch on the same data: prior exposure to other alphabets does not ease learning a new one. Recombining already-known symbols into unfamiliar combinations yields graded, partial success: above chance, short of the seamless transfer an abstract reasoner should show, degrading as combinations diverge from training. Replacing the model’s fixed, per-alphabet output vocabulary with a mechanism that selects among a puzzle’s own visible symbols helps substantially with recombination, but only marginally with transfer to a truly unseen alphabet. These results isolate a specific failure to recognize alphabet-equivalent instances as such, distinct from a general failure to reason, and show several natural interventions narrow, but do not close, this gap.

1 INTRODUCTION

A model that has learned to solve a problem can be said to reason *abstractly* about it if the procedure it applies operates on the problem’s structure, indifferent to the particular form in which that structure is presented. It reasons only *concretely* if what it has learned is tied to the specific surface form it was trained on, instead of to the structure underneath. This distinction is central to any claim that a learned system reasons rather than merely fits patterns in its training distribution, but such claims are frequently made without a test that could actually distinguish the two possibilities. Benchmarks such as SCAN (Lake & Baroni, 2018) and COGS (Kim & Linzen, 2020) test a closely related property, generalizing to unseen *combinations* of familiar components, and have repeatedly found large gaps between in-distribution and compositional-generalization accuracy in standard neural architectures. We study a different, narrower notion of generalization: not new combinations of familiar parts, but the exact same combinatorial structure presented under a purely relabeled vocabulary, which leads to a more basic question: does the model recognize the relabeled instance as the problem it already knows how to solve at all.

We propose a concrete operationalization of this distinction. An abstract reasoner should treat two different instances of the same underlying problem interchangeably: having mastered one, it should transfer to solving the other, since its procedure does not depend on which particular instantiation it happens to face. We instantiate “different instances of the same problem” through the simplest transform that leaves a problem’s structure completely intact while changing its surface form entirely: renaming its symbol alphabet. Two problems P and Q are *equivalent* under this definition

when Q is obtained from P purely by relabeling which symbols represent which underlying roles; nothing about the structure, constraints, or solution changes, only the names used to present it.

This gives more than a single empirical test: a general recipe applicable to any symbolic or combinatorial domain with an analogous notion of relabeling. The recipe has two parts. First, define equivalence via a checkable relabeling, and measure whether a model trained on one instance transfers, unaided, to an equivalent one. Second, use a targeted control to separate two failure modes that a naive transfer evaluation cannot distinguish on its own. A model may fail on a new instance either because its underlying reasoning does not generalize, or simply because the new instance’s symbols were never seen during training, so any parameters associated with them remain at their random initialization regardless of how good the model’s reasoning otherwise is; Sections 4.3–4.4 later test directly whether that alone is enough to prevent any useful signal. We isolate the second explanation with a *weight-transplant* control: after training, we copy the parameters associated with each of P ’s symbols directly onto the parameters for Q ’s corresponding symbols, using the known-by-construction correspondence, with no further training. This tells us how the model would perform on Q if it already knew the correspondence between the two instances’ symbols: precisely the information a naive zero-shot evaluation cannot provide on its own.

We instantiate this recipe concretely on Sudoku puzzles, whose instances can be generated cheaply with a verified unique solution. Training a transformer to solve Sudoku puzzles written in one symbol alphabet, we find that it transfers to an equivalent puzzle in a different alphabet at chance level, performance indistinguishable from a network that was never trained at all, even though the weight-transplant control recovers its original accuracy *exactly*. The model’s procedure does reuse across equivalent instances once it is told how their symbols correspond; what it lacks is any mechanism for discovering that correspondence on its own. We then probe this recognition gap along three further axes: pretraining diversity, an architectural change that removes a specific structural bottleneck, and training-time data diversity at a much larger scale, and show that each narrows, but does not close, the gap, with a further finding that training diversity resolves the gap into two distinct sub-problems with different answers: recombining already-known symbols is solved by sufficient diversity, while transfer to a truly novel symbol is not.

Contributions.

- An operationalized, reusable methodology for testing knowledge transfer between equivalent instances of a symbolic problem, including a weight-transplant control that isolates true reasoning-transfer failure from the confound of never-trained parameters.
- Applied to Sudoku, we show naive transfer across alphabet-equivalent puzzles fails completely (statistically indistinguishable from an untrained network), while the transplant control recovers full performance exactly, demonstrating the failure is one of recognition, not of reusable computation (Section 4.1).
- We show this recognition gap persists across two further natural interventions: pretraining diversity across several known alphabets, and a redesigned architecture that removes a specific structural bottleneck that prevents zero-shot generalization by construction; each narrows but does not close the gap (Sections 4.2–4.3).
- We show training-time data diversity resolves this gap into two distinct sub-problems with different answers: recombining already-known symbols in novel combinations is essentially solved by sufficient training diversity, while transfer to a truly novel, never-seen symbol remains unsolved by every intervention tested (Section 4.4).

2 RELATED WORK

Neural Sudoku solving. Palm et al. (2018) introduced *Recurrent Relational Networks*, which solve Sudoku via iterative message passing between cell-level representations on a graph: the architectural lineage our encoder-based model, and its optional iterative-refinement mechanism, belong to. That work, and subsequent neural combinatorial solvers, focus on solving accuracy within a single fixed vocabulary; we instead hold solving accuracy on the training vocabulary fixed near ceiling throughout, and ask what happens under a symbol relabeling that leaves the underlying combinatorial problem completely unchanged.

Compositional and systematic generalization. SCAN (Lake & Baroni, 2018) and COGS (Kim & Linzen, 2020) test whether sequence models generalize to novel *combinations* of familiar primitives, and both find persistent, large gaps between in-distribution and compositional accuracy across a range of architectures. Our question is narrower: we do not ask the model to combine familiar parts in new ways, only to recognize the exact same combination it was trained on, wearing a different vocabulary. That we still find a total failure to recognize this, indistinguishable from an untrained network, suggests the difficulty compositional-generalization benchmarks document may in part reflect an even more basic failure to abstract away from surface symbol identity at all.

Symbol binding and variable binding. Webb et al. (2021) is the closest direct precedent for our framing: their *Emergent Symbol Binding Network* uses an indirection mechanism over external memory that lets learned rules generalize near-perfectly to entirely novel filler entities, given only a handful of examples, because its computation operates on relations between representations instead of their identity. Altabaa et al. (2024) formalize a version of this idea for Transformers directly, using a relational cross-attention mechanism architecturally constrained to depend only on similarity between representations, never their content. Our Section 4.3 pointer-based redesign is a much simpler, less architecturally constrained step in the same direction: removing a fixed, identity-keyed output vocabulary instead of building in an explicit relational bottleneck. Our results (helping substantially with one sub-problem, only marginally with the other) suggest the stronger, more explicit mechanism of Altabaa et al. (2024) may be necessary where our simpler intervention is not sufficient (see Section 4.4 and Section 5).

Pointer and copy mechanisms. Our Section 4.3 architecture, which predicts a blank cell’s value by selecting among the symbols visible elsewhere in the same puzzle instance instead of classifying into a fixed global vocabulary, is a direct application of Pointer Networks (Vinyals et al., 2015) and pointer-generator copy mechanisms (See et al., 2017) to this setting. Both were originally motivated by output spaces that vary with the input (sorting an input sequence; naming a rare word from the source document) instead of by the alphabet-equivalence question we study; we use the same mechanism for a different purpose, to remove the specific structural obstacle (a per-alphabet output vocabulary) that makes zero-shot transfer to a new alphabet impossible by construction in the naive classifier design.

Symbolic mechanisms in pretrained language models. Contemporaneously with this direction of work, Yang et al. (2025) identify an emergent symbolic architecture inside pretrained large language models (symbol abstraction, symbolic induction, and retrieval heads operating on abstract variables instead of raw tokens) that supports abstract reasoning in a related sense. Our study is complementary: instead of probing an existing large pretrained model, we train small transformers from scratch under fully controlled data, isolating the specific alphabet-relabeling manipulation from every other confound a pretrained model’s training history would introduce.

Positioning. To our knowledge, the specific combination we use, defining instance equivalence via a checkable symbol relabeling, together with a weight-transplant control that isolates true reasoning-transfer failure from the confound of never-trained parameters, has not been used together in this form, though each piece draws on the established ideas above.

3 PROBLEM SETUP AND METHOD

3.1 TASK AND EQUIVALENCE

We use Sudoku as a concrete, verifiable instance of the general setup described in Section 1: a 9×9 grid, partitioned into nine 3×3 boxes, must be filled with symbols such that each symbol appears exactly once in every row, column, and box. A *puzzle* is a partial assignment (some cells given, the rest blank) with a unique completion satisfying these constraints; a *solution* is that completion. Crucially, the constraint structure refers only to *repetition* of symbols, never to their identity: any bijective relabeling of a valid solution’s 9 symbols yields another valid solution to the correspondingly relabeled puzzle. This is what makes Sudoku a clean instance of our general definition: an *alphabet* is an injective map from the 9 canonical digit roles $\{1, \dots, 9\}$ to 9 distinct token ids, and

two puzzles are equivalent exactly when one is the other’s board relabeled under a different alphabet, with the blank symbol (token 0) fixed across all alphabets.

3.2 DATA

We generate 200,000 puzzles once, in a canonical alphabet (token ids 1–9), each with 40 given clues and a solution verified unique by constraint propagation with backtracking. Every other alphabet used anywhere in this paper is a lazy relabeling of this same base set, so all reported comparisons across alphabets are guaranteed to be over *identical* underlying puzzles. We use a fixed 90/5/5 train/val/test split (180,000 / 10,000 / 10,000 puzzles), with the same puzzle indices reused across every alphabet.

Each alphabet is a disjoint block of 9 token ids: the k -th letter of the alphabet (A, B, C, ...) is assigned the token-id offset $9(k - 1)$, so alphabet A is the canonical alphabet itself, B is a +9 shift, and so on. One letter, which we call E, is reserved held out: it is never included in any pretraining pool in any experiment in this paper, so it consistently serves as the truly-unseen-symbol test throughout.

3.3 MODELS

Both architectures we study share an encoder body: a token embedding table, additive row/column/box positional embeddings, and a standard pre-norm Transformer encoder (8 layers, $d_{\text{model}} = 384$, 8 attention heads). They differ only in how a blank cell’s encoded representation is turned into a predicted token.

Classifier head (baseline). A linear layer maps each cell’s representation to $9K$ logits, one per digit per alphabet among the K alphabets ever included in training, and predicts the literal token id directly (never a “digit within my own alphabet,” which would presuppose the model already knows its own alphabet’s identity). This head’s size is fixed at training time: a novel alphabet’s 9 logit rows do not exist yet and cannot be added without retraining, which is the structural bottleneck motivating our second architecture (Section 4.3).

Pointer head. For each blank cell, we instead identify the (at most 9) distinct token ids that appear among that specific puzzle *instance*’s own given clues, and predict which of them belongs in the cell. A learned query, computed from the blank cell’s encoded representation, is compared via dot product against each candidate’s raw token-embedding vector (the same embedding table used on the input side; no separate output projection exists), and the resulting scores are softmaxed over the present candidates. The output space is bounded by the board (at most 9 slots) regardless of how many alphabets have ever been seen during training. A puzzle’s target is only well-defined for a blank cell if the correct digit for that cell is itself among that puzzle’s own givens, not merely somewhere in the completed grid; measured directly on our generator, 0.80% of puzzles violate this. We filter such puzzles out for training, so every training target is well-defined, and deliberately leave them in for evaluation, so reported numbers reflect this real (if small) architectural limitation instead of hiding it.

Both architectures optionally support unrolling the shared encoder for several iterations, feeding back each step’s prediction for still-blank cells as if it were a new given (RRN/IREM-style); all experiments in this paper use a single pass ($T = 1$).

3.4 THE WEIGHT-TRANSPLANT CONTROL

After training a model on alphabet A, we can ask how it would perform on an equivalent puzzle in alphabet B *if it already knew* the digit-for-digit correspondence between A and B’s tokens, independent of whether B’s own parameters ever received a training signal. For the classifier head, we do this by copying, for every digit $d \in \{1, \dots, 9\}$, the trained embedding row and output-head row/bias for A’s token for d directly onto B’s token for d (a plain tensor copy, no gradient step, no retraining) before evaluating on B. (The pointer head needs no analogous control: it never has per-alphabet output parameters to copy in the first place, which is precisely its motivation.) This isolates two failure modes a naive cross-alphabet evaluation cannot distinguish on its own: failure because the model’s

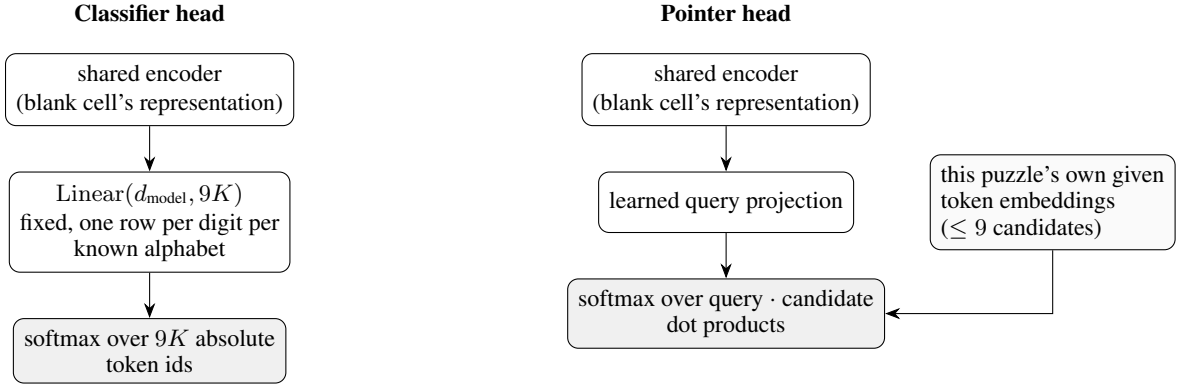


Figure 1: The two architectures share an encoder body and differ only in how a blank cell’s representation becomes a predicted token. The classifier head (left) scores a fixed set of $9K$ absolute token ids, requiring dedicated trained rows for every alphabet ever seen. The pointer head (right) instead scores only the token ids actually present as givens in that specific puzzle instance, so its output space never depends on how many alphabets were seen during training.

reasoning does not generalize across alphabets, versus failure simply because B’s parameters were never trained at all.

3.5 METRICS

We report *cell accuracy* (fraction of blank cells filled correctly), *exact match rate* (fraction of boards filled completely correctly), and *valid rate* (fraction of boards that are legal completions, meaning no repeated symbol in any row, column, or box, consistent with the given clues, regardless of whether they match one specific labeling). For the classifier head, computing cell accuracy requires restricting the argmax to the evaluated alphabet’s own 9 logits: an unrestricted argmax lets other alphabets’ classes compete, and once the transplant control makes two alphabets’ rows numerically identical for the same digit, argmax’s tie-breaking would otherwise systematically favor whichever alphabet has the lower token offset, which does not reflect real model behavior. Valid rate is the primary metric whenever a specific ground-truth correspondence cannot be assumed known (Sections 4.3–4.5): since Sudoku’s solutions are equivariant to symbol relabeling, there are $9!$ equally valid labelings of any completed grid, so exact match against one arbitrarily chosen labeling would unfairly penalize a model that solved the puzzle correctly under a different, equally valid one.

3.6 TRAINING PROTOCOL

All models are trained with AdamW, batch size 256, for 20 epochs unless otherwise noted, on a single A100 GPU per run. Full hyperparameters are listed in the Appendix.

4 EXPERIMENTS AND RESULTS

Unless noted otherwise, every experiment below uses the data, model, and metrics of Section 3, trained on a single A100 GPU per run.

4.1 DOES THE MODEL DISCOVER THE EQUIVALENCE ON ITS OWN?

We train the classifier head on alphabet A only and, separately, on alphabet B only, plus a reference model trained on the union of both. Each single-alphabet model is evaluated three ways: on its own alphabet (*self*); on the other alphabet, unaided (*naive cross*); and on the other alphabet with the weight-transplant control applied (*transplanted cross*).

Naive cross-alphabet transfer lands exactly at the random-init floor: not somewhat worse than self-eval, but statistically indistinguishable from a network that was never trained at all. The transplant

Model	Eval alphabet	Mode	Cell acc.	Exact match
A	A	self	99.41%	92.2%
A	B	naive cross	11.28%	0%
A	B	transplanted cross	99.41%	92.2%
B	B	self	99.46%	92.3%
B	A	naive cross	10.95%	0%
B	A	transplanted cross	99.46%	92.3%
random init	(either)	floor	11.10–11.17%	0%

Table 1: Naive cross-alphabet transfer lands exactly at the random-init floor; the transplant control recovers the self-eval number exactly.

control recovers the self-eval number to four significant figures, with no retraining. The only thing separating 11% from 99% is knowing the symbol correspondence, not some deeper representational gap: the model’s solving competence is bound to the specific tokens it trained on, and it does not spontaneously discover that a renamed puzzle is the same problem it already knows how to solve.

4.2 DOES MORE ALPHABET DIVERSITY HELP?

Experiment 4.1 used one alphabet per model. We ask two further questions with a classifier pre-trained on $K = 6$ alphabets (A,B,C,D,F,G; E held out), 60,000 puzzles per alphabet, 20 epochs (99.57% cell accuracy across all six by the end of training).

Few-shot adaptation. For held-out E, we compare two conditions given $n \in \{100, 1000\}$ labeled E-puzzles, 3 seeds each: *finetune* freezes the entire pretrained body and lets only E’s own embedding and output rows adapt; *scratch* trains a fresh, unfrozen model on the same small data.

Condition	$n = 100$	$n = 1000$
zero-shot (no fine-tuning)	11.8%	–
finetune (frozen body)	26.8%	39.5%
scratch (unfrozen)	26.5%	69.2%

Table 2: Mean cell accuracy on E’s held-out test split. At $n = 1000$ the finetune condition is nearly flat across all 3 seeds.

At $n = 100$ the two conditions are indistinguishable. At $n = 1000$, training from scratch clearly beats reusing the frozen pretrained body, and the finetune number barely moves across seeds, a real ceiling rather than noise. If the frozen body were a portable, alphabet-agnostic solving procedure, reusing it and only relearning vocabulary should have been easier than relearning everything from the same data. It was not: the shared network body is entangled with the specific alphabets it was trained on, not a clean, reusable abstraction.

Mixed-alphabet compositional test. Using the same $K = 6$ checkpoint, we recombine the 6 known alphabets instead of testing a new one: each of the 9 digit roles independently draws its symbol from a possibly different known alphabet, so the resulting combination was never presented as one coherent alphabet during training, even though every individual symbol is well learned.

Trial	Cell accuracy
control (all digits from alphabet A)	99.49%
maximally mixed (9 digits, 6 alphabets, cycled)	44.5%
10 random mixes	23.2%–70.8% (mean 45.4%, s.d. 12.4%)

Table 3: Recombining known alphabets: well above chance, well below the 99% ceiling, with accuracy tracking how coherent the specific recombination is.

Accuracy is far above the roughly 11% chance level but far below the 99% ceiling, and it tracks how coherent the specific mix is: dominated by one alphabet scores high, spread evenly across many scores low. Both results point the same way: the model has partial, graded compositional ability instead of a clean, disentangled binding of each symbol to its meaning.

4.3 DOES REMOVING THE OUTPUT-VOCABULARY BOTTLENECK HELP?

We repeat the $K = 6/60,000$ -per-alphabet/20-epoch protocol with the pointer head (Section 3) instead of the classifier head, same held-out E, for direct comparability.

Alphabet	Cell accuracy	Valid rate
A, B, C, D, F, G (each)	99.3%	92.5%–93.0%

Table 4: In-distribution gate: matches the classifier’s ceiling; the architecture change costs nothing on the easy case.

Metric	Held-out alphabet E
Cell accuracy	14.47%
Exact match rate	0%
Valid rate	0%

Table 5: Zero-shot transfer to a never-seen alphabet.

Valid rate is the more decisive number here. There are $9!$ equally valid relabelings of any solution, so a model that had learned to produce some internally consistent completion, even one that does not match our specific labeling, would show a nonzero valid rate; zero out of 10,000 boards rules that out entirely. The 14.47% cell accuracy is a small but statistically real signal above the roughly 11.1% chance floor (with about 410,000 blank test cells, the gap is far larger than sampling noise would allow), unlike the classifier’s flat 11.8% in Section 4.2. Removing the fixed-vocabulary bottleneck was necessary but not sufficient: it gave a small edge, not the clean transfer we set out to test for.

Trial	Classifier	Pointer	Δ
control (all-A)	99.5%	99.4%	~ 0
maximally mixed cycle	44.5%	34.9%	−9.6pp
random 0	70.8%	59.5%	−11.3pp
random 1	53.1%	75.8%	+22.7pp
random 2	42.5%	46.5%	+4.0pp
random 3	38.7%	45.3%	+6.5pp
random 4	23.2%	43.0%	+19.8pp
random 5	39.2%	76.3%	+37.1pp
random 6	45.6%	42.7%	−2.9pp
random 7	54.0%	45.9%	−8.0pp
random 8	41.1%	34.4%	−6.7pp
random 9	46.3%	58.7%	+12.4pp
mean of 10 random trials	45.4% (s.d. 12.4)	52.8% (s.d. 14.3)	+7.4pp

Table 6: Same mixed-alphabet protocol as Table 3, same seeds, classifier versus pointer head.

A real average improvement, and the pointer head helps most on the classifier’s worst trials: the two largest gains land on the classifier’s two lowest scores, and the floor across all trials rises from 23.2% to 34.4%. But the improvement is uneven; 4 of the 11 non-control trials get worse, including the most systematic one (maximally mixed cycle). The architecture change helps more with the easier question (recombining known symbols) than the harder one (a truly novel symbol), in both cases by a modest instead of decisive margin.

One candidate explanation for why the pointer head shows any positive zero-shot signal at all, where the classifier shows none: the classifier’s failure was total because no mechanism let gradient reach an untrained output row. The pointer head’s E-alphabet embeddings are also untrained, at their random initialization, but every occurrence of a given E-symbol within one puzzle uses the identical random vector, and detecting that two positions hold the same vector is a property of equality between representations, not of their content. A “do not repeat a symbol within this row, column, or box” mechanism built on that kind of comparison could plausibly fire correctly even for never-trained tokens. This would predict the observed pattern (a weak but real signal, far from a solved task) but has not been verified directly; Section 5 discusses it as a candidate for future interpretability work, not a settled finding.

4.4 DOES MORE TRAINING DIVERSITY CLOSE THE GAP?

Section 4.3 left a real but small zero-shot improvement unexplained. Two candidate explanations motivate this experiment: the $K = 6$ pretraining alphabets were each repeated across 20 epochs, letting the model partially memorize those 6 specific combinations; or something more structural limits the pointer head regardless of training diversity. We test the first explanation directly, at low cost, reusing the pointer architecture from Section 4.3 without any code change to the model itself: instead of 6 fixed alphabets, every single training example draws an independently random digit-to-letter mapping, fresh, from a pool of 25 non-E letters (each of the 9 digit roles independently, not one random coherent alphabet per puzzle). No fixed combination is ever repeated for the model to memorize.

Training reaches 98.41% cell accuracy on the held-out (also randomly mixed) validation set by epoch 19, over 20 epochs of $\sim 178,560$ fully-represented puzzles (no repetition needed, since every epoch’s pass already gives each puzzle a fresh mix).

Metric	All six known alphabets
Cell accuracy	98.2%
Valid rate	76.9%–78.0%

Table 7: In-distribution gate, testing each alphabet as a pure coherent block, as in Table 4.

This is notably lower than Section 4.3’s fixed-alphabet pointer (99.3% cell, 92.5–93.0% valid). This model almost never saw a pure single-alphabet puzzle during training, since each of the 9 digit roles is drawn independently from 25 letters, so a coherent alphabet is itself a rare, close-to-out-of-distribution case for it, unlike for the fixed-alphabet-trained model.

Metric	Fixed alphabets (4.3)	Random mix (this section)
Cell accuracy	14.47%	15.85%
Valid rate	0%	0%

Table 8: Zero-shot transfer to held-out alphabet E.

A small additional bump in cell accuracy, but valid rate stays at exactly 0%: the same categorical failure as before, now even under a training distribution spanning 25 letters with a fresh random combination on every example. Training diversity, at least at this scale, does not meaningfully close the zero-shot gap.

Metric	Fixed alphabets (4.3)	Random mix (this section)
Control (all-A)	99.4%	98.2%
Maximally mixed cycle	34.9%	98.3%
Range, 10 random trials	34.4%–76.3%	98.0%–98.3%
Mean, 10 random trials	52.8% (s.d. 14.3)	98.2% (s.d. 0.1)
Valid-rate range, all 12 trials	0%–92.6%	76.3%–79.5%

Table 9: Same mixed-alphabet protocol and seeds as Tables 3–6.

The trial-to-trial variance seen in every earlier condition (classifier 23–71%, fixed-alphabet pointer 34–76%) has essentially vanished: every one of the 12 trials, from the coherent control to the most aggressively mixed random draw, now scores within a 0.3 percentage-point band. Random-mix training exposes the model to an enormous number of recombinations of the same 25 known letters; by the end of training it has effectively seen every letter co-occurring with every other letter in nearly every digit-role position, many times over, so no new combination of already-known letters is truly novel to it anymore. But alphabet E was never in that pool at all, so recombining other letters injects no information into E’s specific embedding, which remains exactly as untrained as it always was. Training diversity fixes whether the model has seen a given combination of known symbols before; it does not fix whether the model has ever seen a given symbol at all. This training regime also trades some peak performance (coherent-alphabet valid rate drops from about 93% to 77–79%) for a large gain in consistency across the whole combinatorial space (valid-rate range collapses from 0–93% to 76–80%), an ordinary specialist-versus-generalist trade-off.

4.5 DISENTANGLING POOL SIZE FROM MIXING EXPOSURE

Section 4.4 changed two variables relative to Section 4.3 at once: the alphabet pool grew from 6 to 25 letters, and training switched from never mixing letters within a puzzle to always mixing them. This experiment holds pool size fixed at Section 4.3’s level (the identical 6 letters, and an identical vocabulary size) while giving it Section 4.4’s always-mixed training format, to find out which variable actually drove the variance collapse.

Metric	$K=6$, never mixed	$K=25$, always mixed	$K=6$, always mixed
Range, 10 random trials	34.4%–76.3%	98.0%–98.3%	98.5%–98.7%
Valid-rate range, 12 trials	0%–92.6%	76.3%–79.5%	81.0%–84.3%

Table 10: Mixed-alphabet compositional test under three training conditions.

With only 6 letters, the same pool that showed 34–76% variance in Section 4.3, training with always-mixed puzzles reproduces the same tight, uniform band seen with 25 letters in Section 4.4, in fact slightly better (98.5–98.7% versus 98.0–98.3%, and 81–84% versus 76–80% valid rate), plausibly because learning capacity is not spread across as many distinct symbols. Going from 6 to 25 letters bought essentially nothing extra for this sub-problem. The variable that matters is whether training ever exposes the model to within-puzzle mixing at all, not how large the alphabet pool is.

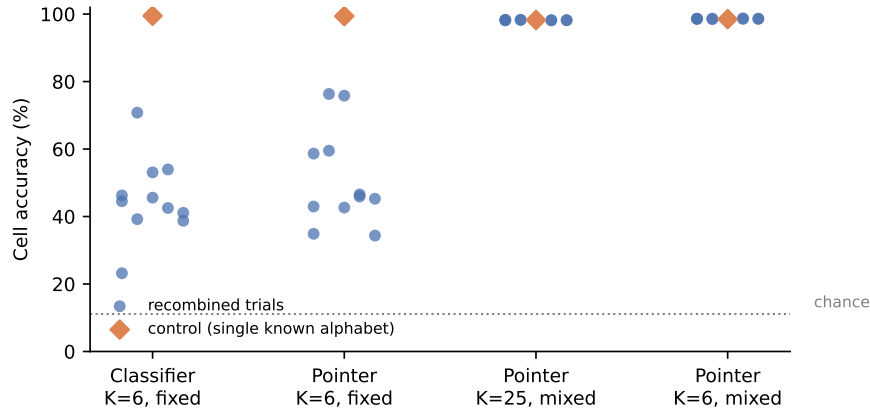


Figure 2: Per-trial cell accuracy on the mixed-alphabet compositional test (Tables 3, 4, 9, 10), across the four conditions that were evaluated on it. Each point is one of the 11 recombination trials (the maximally mixed cycle plus 10 random draws); diamonds mark the coherent single-alphabet control. Training that never mixes alphabets within a puzzle (the two leftmost conditions) leaves accuracy highly sensitive to which specific recombination is tested; training that always mixes them (the two rightmost conditions) collapses that variance almost entirely, regardless of how many letters the training pool contains.

Figure 2 makes this collapse visible across the full arc of conditions tested: wide, trial-dependent spread under never-mixed training, tight clustering near the ceiling under always-mixed training, in both the 25-letter and 6-letter pools.

On zero-shot transfer to held-out E, this condition reaches 11.46% cell accuracy and 0% valid rate. The small positive zero-shot signal seen in Sections 4.3 (14.47%) and 4.4 (15.85%) is essentially absent here, close to the roughly 11.1% chance floor. Valid rate stays at exactly 0% across all three conditions, so the headline conclusion is unchanged either way, but this leaves open whether the weak signal in the other two conditions depends on genuine pool diversity or is simply run-to-run noise: each condition here is a single training run, so we flag this as a real open question instead of a resolved finding (Section 6).

Figure 3 places all five zero-shot-on-E numbers side by side: cell accuracy moves within a narrow, low band across every intervention, while valid rate never leaves zero.

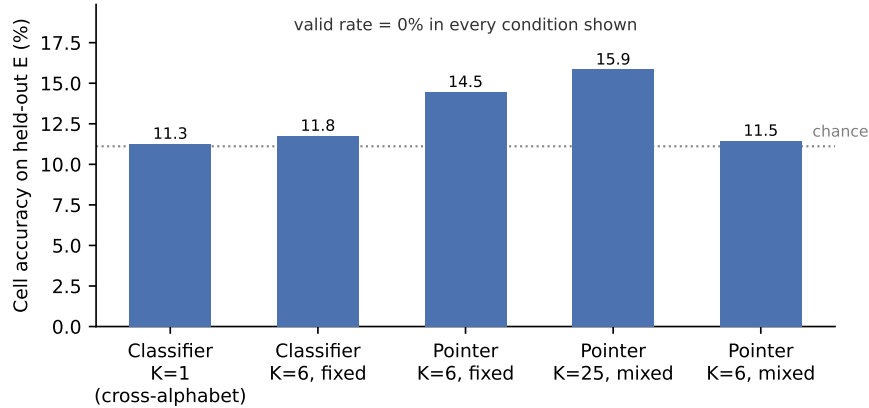


Figure 3: Cell accuracy on held-out alphabet E, zero-shot, across every condition tested (Sections 4.1–4.5). No condition rises meaningfully above the 1/9 chance floor, and valid rate is exactly 0% throughout: none of pretraining diversity, an alphabet-agnostic output mechanism, or exposure to within-puzzle mixing produces a single fully consistent completion on a symbol the model never trained on.

5 DISCUSSION

A single failure mode appears from every angle we tested, and it resolves into two sub-problems with different answers.

Recombining known symbols is solved by training diversity, specifically by mixing exposure instead of pool size. Sections 4.2 and 4.3 both showed graded, partial degradation on recombined-alphabet puzzles under training that never mixed alphabets within a puzzle; Section 4.4 showed that degradation essentially vanishes once training always mixes a large pool of letters; Section 4.5 located why, by reproducing the same near-perfect uniformity with only 6 letters, as long as training always mixes them. This part of the entanglement was, in the end, a matter of what the training distribution exposed the model to, and a cheap property to supply.

Transfer to a symbol never seen at all is not solved by anything we tried. Neither the architectural change in Section 4.3 nor either training-diversity condition in Sections 4.4–4.5 moved held-out alphabet E’s valid rate off exactly 0%. Cell accuracy’s small residual signal is less consistent than it first appeared: it rose from 11.8% in Section 4.2’s classifier baseline to 14.47% in Section 4.3 to 15.85% in Section 4.4, but fell back to 11.46%, close to chance, in Section 4.5. Whether that pattern reflects a real dependence on pool diversity or simply run-to-run variation across a narrow band is not settled by a single run per condition; either way, valid rate never once moved, so nothing tried so far gives the model a way to discover what an unseen symbol means well enough to produce a single fully valid board.

The result underlying both halves of this picture is Section 4.1’s transplant control: the model’s procedure reuses across equivalent instances exactly, once it is told the symbol correspondence. Read against the framing in Section 1, this sharpens what “concrete” actually means for these models. It is not that the computation itself fails to generalize; handed the correspondence, it transfers perfectly. What remains bound to specific training instances is the recognition step, the part that would let the model notice, on its own, that a new instance calls for the procedure it already has. Every intervention in this paper either supplies more of that recognition capacity directly (the transplant control) or tries to induce it indirectly through architecture or data, and only the indirect routes were tested at scale; none closed the harder half of the gap.

One candidate account for the small residual signal in Sections 4.3 and 4.4, and its absence in Section 4.5, is the vector-equality mechanism sketched in Section 4.3: a “do not repeat this symbol nearby” detector that only needs two representations to be compared for equality, not for their content to be known, could transfer to never-trained embeddings by accident, and could plausibly

depend on how much practice the model gets comparing many distinct vectors during training. This is consistent with the pattern observed but has not been tested directly; confirming it would need an interpretability probe comparing attention or compatibility patterns on held-out E's cells against the same puzzle in a known alphabet, which we leave to future work.

Given that no data-side intervention touched the harder half of the gap, the most promising next step is architectural: a mechanism that constrains computation to operate on similarity between representations by construction, instead of leaving that as one option among others the way the pointer head in Section 4.3 does. Abstractors (Altabaa et al., 2024) implement exactly this idea for Transformers through relational cross-attention, and are a natural candidate for closing what training-side changes could not.

6 LIMITATIONS

Every model in this paper is a small transformer trained from scratch under fully controlled data; we make no claim about how these findings would look in a large pretrained model, whose training history already spans far more surface variation than any single alphabet-relabeling manipulation could add. Section 2 discusses one recent result suggesting a related distinction is visible inside pretrained LLMs too (Yang et al., 2025), but we did not test this directly.

We deliberately study one domain, Sudoku, instead of several; this keeps every comparison across experiments over identical underlying puzzles, at the cost of not yet knowing how much of what we find is specific to this domain's particular constraint structure. Within that domain, we study one equivalence transform, relabeling the symbol alphabet, out of several a puzzle could plausibly support (permuting rows and columns, or changing notation, for instance); whether these other transforms produce the same recognition gap is untested.

The pointer head (Section 4.3) has a structural blind spot: it can only predict a digit whose token already appears among a puzzle's own givens, and 0.80% of puzzles from our generator have at least one digit that does not. We measured this directly instead of assuming it away, filtered it out of training, and left it in for evaluation, but the architecture is mechanically unable to solve those puzzles regardless of how well it has otherwise learned.

Most training conditions in Section 4 are a single run; the few-shot sweep in Section 4.2 is the exception, at 3 seeds. The residual zero-shot cell-accuracy signal discussed in Section 5 moved across a narrow, non-monotonic range between conditions, and we cannot currently rule out that this particular pattern is run-to-run variation instead of a real effect of pool diversity; every condition's valid rate, our primary and more decisive metric, is unaffected by this uncertainty, since it stayed at exactly 0% throughout.

7 CONCLUSION

We gave a checkable operationalization of a question usually left informal, whether a model's competence transfers between instances of the same problem that differ only in surface symbols, together with a control that separates a true failure to transfer reasoning from the simpler fact that some parameters were never trained. Applied to Sudoku, naive transfer across a relabeled alphabet fails completely, while the transplant control recovers full performance exactly: the model's procedure is reusable, but nothing in ordinary training gives it a way to recognize, on its own, that a new instance calls for it. Across several attempts to close that gap, one sub-problem, recombining symbols the model already knows, turned out to be a matter of training diversity and is now essentially solved; the other, transferring to a symbol the model has never encountered at all, was untouched by every architectural and data-side intervention we tried. That split result is itself the clearest signal for where to look next: an architecture that constrains its computation to operate on relations between representations, instead of merely allowing it to, is now the best-motivated remaining direction.

REFERENCES

Awni Altabaa, Taylor Webb, Jonathan Cohen, and John Lafferty. Abstractors and relational cross-attention: An inductive bias for explicit relational reasoning in Transformers. In *International*

- Conference on Learning Representations, 2024.
- Najoung Kim and Tal Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9087–9105, 2020.
- Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International Conference on Machine Learning*, pp. 4487–4499, 2018.
- Rasmus Berg Palm, Ulrich Paquet, and Ole Winther. Recurrent relational networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1073–1083, 2017.
- Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, pp. 2692–2700, 2015.
- Taylor W. Webb, Ishan Sinha, and Jonathan D. Cohen. Emergent symbols through binding in external memory. In *International Conference on Learning Representations*, 2021.
- Yukang Yang, Declan Campbell, Kaixuan Huang, Mengdi Wang, Jonathan Cohen, and Taylor Webb. Emergent symbolic mechanisms support abstract reasoning in large language models. In *International Conference on Machine Learning*, 2025.

A APPENDIX

A.1 HYPERPARAMETERS

Data	
Puzzles generated	200,000, once, canonical alphabet
Given clues per puzzle	40
Split	180k / 10k / 10k train/val/test, fixed indices
Model (both heads)	
Encoder	8-layer pre-norm Transformer, $d_{\text{model}} = 384$, 8 heads
Feedforward dimension	1536
Dropout	0.1
Positional embeddings	additive row/column/box embeddings
Unrolled iterations	1 (architecture supports more; unused here)
Pointer query projection	$\text{Linear}(d_{\text{model}}, d_{\text{model}})$, always used
Training	
Optimizer	AdamW, $\text{lr } 3 \times 10^{-4}$, weight decay 0.01
Batch size	256
Epochs, single/union alphabet (4.1)	30
Epochs, K -alphabet pretraining (4.2–4.5)	20
Epochs, few-shot finetune/scratch sweep (4.2)	150
Seeds, few-shot finetune/scratch sweep	3 (0, 1, 2) per (n , condition)
Seeds, all other conditions	1

Table 11: Hyperparameters used across all experiments. Section labels indicate where each setting is first used.

For the frozen-body condition of Section 4.2, the shared encoder and positional embeddings are frozen via `requires_grad_(False)`, and the token-embedding and classifier-head rows belonging to alphabets other than the target one are kept fixed by masking their gradient at the moment `autograd` produces it (a backward hook), instead of by zeroing gradients after the fact. This matters because AdamW’s weight decay shrinks every parameter on every step regardless of its gradient, so a masked-gradient row would still drift under nonzero weight decay; the finetuning optimizer therefore uses weight decay 0 for this condition only, overriding the value in Table 11.

A.2 COMPUTE

All experiments ran on Leonardo (CINECA EuroHPC), a single NVIDIA A100 GPU per job. For Section 4.1’s single- and union-alphabet training, we have measured wall-clock time directly: 52 minutes each for the two single-alphabet runs and 1 hour 43 minutes for the union run (≈ 3.5 GPU-hours together). For every other job, Table 12 reports the wall-clock budget requested at submission time instead of measured usage; where we do have measured numbers (Section 4.1), actual usage came in well under the requested budget, so the totals below should be read as an upper bound on total compute, not an exact accounting.

Job	Requested wall-clock
Data generation (one-time)	1h00m
Single-alphabet training $\times 2$, union training (Section 4.1)	2h00m + 2h00m + 3h00m
Cross-alphabet evaluation	0h30m
$K=6$ classifier pretraining (Section 4.2)	2h00m
Few-shot finetune/scratch sweep	1h30m
Mixed-alphabet evaluation	0h20m
$K=6$ pointer pretraining (Section 4.3)	2h00m
Pointer zero-shot and mixed-alphabet evaluation	0h30m + 0h20m
$K=25$ random-mix pointer pretraining (Section 4.4)	1h30m
Random-mix zero-shot and mixed-alphabet evaluation	0h30m + 0h20m
$K=6$ always-mixed pointer pretraining (Section 4.5)	1h30m
$K=6$ always-mixed zero-shot and mixed-alphabet evaluation	0h30m + 0h20m
Total requested (upper bound)	≈ 20 GPU-hours

Table 12: Requested wall-clock per SLURM job, one A100 GPU each. Measured usage where available (Section 4.1) came in roughly half of the requested budget.

A.3 AN ABLATION WE DID NOT RUN

The pointer head’s query is computed via a learned linear projection of the blank cell’s representation before it is compared against candidate embeddings (Section 3). The implementation also supports skipping this projection and comparing the raw encoder output directly against candidate embeddings (`use_query_projection=False`), a cheaper variant motivated by the pointer-network and copy-mechanism literature we build on (Vinyals et al., 2015; See et al., 2017). Every result reported in this paper uses the learned projection; the raw variant was never trained or evaluated, so we cannot say whether it would change the zero-shot transfer results in Sections 4.3–4.5. We leave it as an open, cheap ablation for future work.